

Implementacja sterownika USB 2.0 Full-Speed Mass Storage Class na mikrokontrolerze RISC-V

(Implementation of a USB 2.0 Full-Speed Mass Storage Class driver
on a RISC-V microcontroller)

Kamil Zdancewicz

Praca licencjacka

Promotor: dr Marek Materzok

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

19 czerwca 2026

Streszczenie

Protokoły komunikacyjne od prostych jak USART z niewieloma przewodami do stabilnego i odpornego na zakłócenia CAN-a lub *uniwersalny*, pozwalający na kontrolowany przesył danych i zasilania USB odpowiadają za komunikację pomiędzy urządzeniami lub modułami pewnego systemu. Omawiany w tej pracy protokół USB, w szczególności transfery typu Bulk-Only i analiza potoku logicznego na przykładzie urządzenia Mass Storage Class, pozwoli na niskopoziomowy wgląd w komunikację z peryferiami oraz zachowanie na granicy protokołu (USB) a interfejsu (SCSI). Teoria ma podłoże praktyczne w rzeczywistej implementacji kontrolera USB na mikrokontrolerze symulującego prosty nośnik pamięci masowej USB z dużą swobodą na modyfikacje niezależne od protokołu takie jak backend pamięci.

Communication protocols ranging from simple USART with few wires to the stable and fault-tolerant CAN, or *universal*, fine data and power transmission controlling USB facilitate communication between devices or modules within a given system. The USB protocol discussed in this paper, specifically Bulk-Only transfers and logical stream analysis using the Mass Storage Class device as an example, will provide a low-level insight into communication with peripherals and the interconnectedness between the protocol (USB) and the interface (SCSI). The theory is grounded in the practical implementation of a USB controller on a development board simulating a simple USB mass storage device, with flexibility for protocol-independent modifications such as the memory backend.

Spis treści

1. Wstęp	9
1.1. Perspektywa	9
1.2. Kontekst sprzętowy	9
1.3. Cel i zakres	10
2. Platforma sprzętowa i narzędziowa	11
2.1. Procesor i pamięć	12
2.2. Kontroler USB	12
2.3. Inne peryferia	13
3. Protokoły komunikacyjne	15
3.1. USART	15
3.2. SPI i I ² C	16
3.3. CAN	17
3.4. USB	18
3.4.1. Topologia	18
3.4.2. Przesył danych	18
3.4.3. Punkty końcowe	20
4. Struktury i transfery danych USB	21
4.1. Deskryptory i struktura logiczna	21
4.2. Transfery kontrolne i potok sterujący	23
4.3. Transfery masowe i protokół Bulk-Only Transport	24
4.3.1. Architektura protokołu Bulk-Only Transport	24

5. Komendy SCSI	27
5.1. SPC-3	27
5.1.1. Komendy specyfikacji SPC-3	28
5.2. SBC-2	29
5.2.1. Komendy specyfikacji SBC-2	29
5.3. SSC-2	30
6. Diagnostyka, testowanie i weryfikacja	31
6.1. Środowisko deweloperskie i narzędzia	31
6.1.1. Środowisko deweloperskie	31
6.1.2. Narzędzia	31
6.2. Efekt i testy	32
6.2.1. Zachowanie sterowników	33
6.2.2. Testy funkcjonalne	35
6.2.3. Efekt	35
6.3. Wyzwania	36
7. Zakończenie	37
7.1. Możliwości rozwoju	38
Bibliografia	39

Wykaz skrótów

USART	Universal Synchronous and Asynchronous Receiver-Transmitter
SPI	Serial Peripheral Interface
I²C	Inter-Integrated Circuit
CAN	Controller Area Network
USB	Universal Serial Bus
BOT	Bulk-Only Transport
LBA	Logical Block Addressing
EP	Endpoint
TX	Transmit / Transmitter
RX	Receive / Receiver
ACK	Acknowledgement
NAK	Negative Acknowledgement
USBFS	USB Full Speed
USBD	USB Full-speed Device Controller
MSC	Mass Storage Class
IRQ	Interrupt Request
RCC	Reset and Clock Control
SCSI	Small Computer System Interface
SBC-2	SCSI Block Commands - 2
SPC-3	SCSI Primary Commands - 3
SSC-2	SCSI Stream Commands - 2
RISC	Reduced Instruction Set Architecture
HAL	Hardware Abstraction Layer
CBW	Command Block Wrapper
CSW	Command Status Wrapper
CBWCB	Command Block Wrapper Command Block

ISA	Instruction Set Architecture
CSR	Control and Status Register
SRAM	Static Random-Access Memory
PFIC	Programmable Fast Interrupt Controller
VTF	Vector Table Free
ISR	Interrupt Service Routine
NRZI	Non-Return-to-Zero, Inverted
OTG	On-The-Go
GPIO	General-Purpose Input/Output
GND	Ground
CLK	Clock
SDA	Serial Data
SCL	Serial Clock
SDIO	Secure Digital Input Output

Rozdział 1.

Wstęp

W dzisiejszym świecie urządzenia pamięci masowej, znane prościej jako pen-drive'y lub dyski zewnętrzne, są traktowane jako naturalne rozszerzenie pamięci urządzeń. Przez płynne i naturalne działanie w zgodzie z dowolnym komputerem łatwość użytku bez wiedzy dotyczącej ich wewnętrznego działania doprowadziła do ich powszechnego przyjęcia jako sprawdzony sposób na przechowywanie i przenoszenie niewielkich ilości danych. Zainteresowało mnie jakimi zasadami rządzą się te urządzenia: jakie są minimalne wymagania oraz jakie techniki są wykorzystywane, aby osiągnąć ten efekt.

1.1. Perspektywa

Osiągnięcie tego wymagało zmiany podejścia do architektury tworzonego oprogramowania z tradycyjnego modelu, w którym program spełnia pewną funkcjonalność w próżni i w pełni kontroluje przepływ sterowania na reaktywny i usługowy tryb działania, zbliżony do dynamiki serwera z klientem. W tym kontekście implementowane urządzenie pomimo stanowienia osobnego systemu od urządzenia nadrzędnego (*hosta*) ma za zadanie spełniać żądania w domenie czasu rzeczywistego jako urządzenie podrzędne (*device*). W związku z tym nieodzownym elementem rozwoju było zrozumienie działania sterownika *hosta* oraz użycie narzędzi pozwalających go nadzorować.

1.2. Kontekst sprzętowy

Do realizacji tego zadania wybrałem mikrokontroler **CH32V305** oparty na architekturze **RISC-V** [7]. Ta architektura opiera się na zasadzie Reduced Instruction Set Architecture (RISC) wraz z otwartością i permissywną licencją, co umożliwia niezależnym twórcom jej implementację bez płacenia należności licencyjnych. W ostatnich latach zyskuje ona znaczenie na rynku europejskim i chińskim dzięki rosnącemu

znaczeniu niezależności od zamkniętych architektur takich jak x86-64 lub AArch64. Pisanie oprogramowania warstwy Universal Serial Bus (USB) na mniej standardowej architekturze wiązało się z użytkowaniem bardzo podstawowego, zapewnionego przez producenta Hardware Abstraction Layer (HAL)-u i ekstensywnym poleganiu na podręczniku obsługi płytki [15].

1.3. Cel i zakres

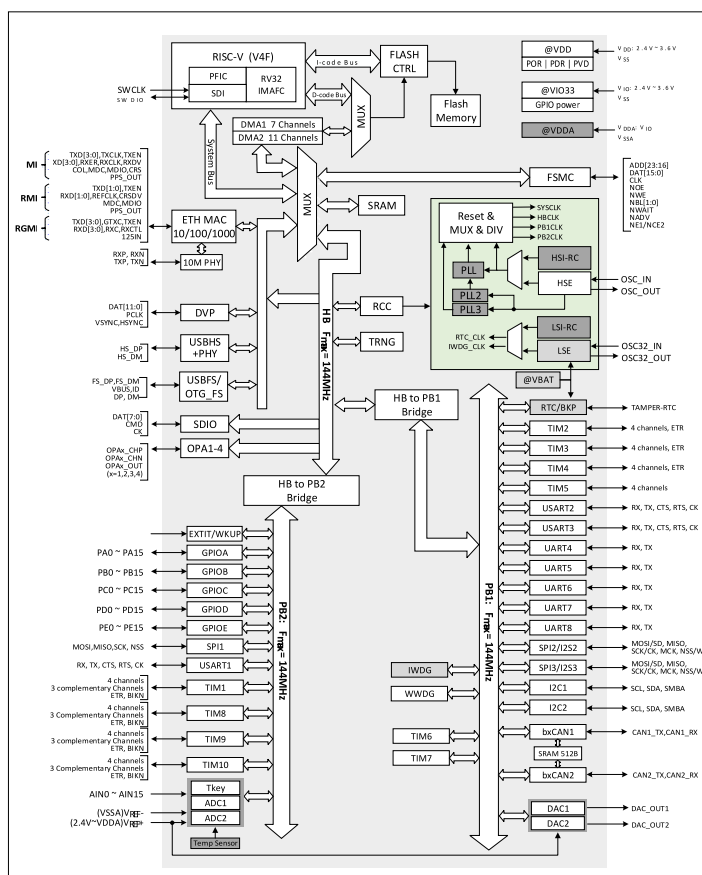
Celem niniejszej pracy było zaprojektowanie, implementacja oraz weryfikacja działania oprogramowania wbudowanego (*firmware*) realizującego funkcjonalność urządzenia USB MSC na tym mikrokontrolerze. Aby to zrealizować, przeszedłem przez następujące kroki:

- Rozpoznanie i niskopoziomowa obsługa wbudowanego kontrolera USB Full Speed (USBFS), Interrupt Request (IRQ) oraz pamięci FLASH.
- Analiza specyfikacji USB 2.0 [26] ze szczególnym uwzględnieniem obsługi pakietów kontrolnych (*Setup*) w fazie enumeracji oraz warstwy transportowej Bulk-Only Transport (BOT).
- Oprogramowanie logiki struktury CBW oraz implementacja wymaganych komend protokołu Small Computer System Interface (SCSI) [9][8][1].
- Weryfikacja i testy polegające na uruchomieniu gotowego urządzenia i sprawdzeniu poprawnego montowania oraz prędkości transferu danych.
- Zapewnienie pełnej abstrakcji obsługi urządzenia.

Rozdział 2.

Platforma sprzętowa i narzędziowa

CH32V305 to mikrokontroler ogólnego użytku z szeroką gamą peryferiów, czyli rozwiązań elektronicznych, które odciążają procesor lub pozwalają mu na interakcję z otoczeniem, w tym rozdziale przedstawiam jego elementy niezbędne do zrealizowania planowanego zadania oraz zewnętrzne narzędzia, które ułatwiły pracę.



Rysunek 2.1: Schemat architektury mikrokontrolerów CH32V30x [15, wykres 1-6].

2.1. Procesor i pamięć

Używany mikrokontroler jest wyposażony w mikroprocesor **QingKe V4F** [16] z bazą **RV32I** – podstawowy 32-bitowy Instruction Set Architecture (ISA) dla liczb całkowitych, dodatkowo wspiera on następujące rozszerzenia zbioru instrukcji:

Rozszerzenie	Znaczenie
M	Mnożenie
A	Instrukcje atomowe
C	Instrukcje skompresowane
F	Liczby zmiennoprzecinkowe
Zicsr	Instrukcje Control and Status Register (CSR)
Zifencei	Spójność cache instrukcji

Tabela 2.1: Rozszerzenia RISC-V

Układ posiada 32 KiB pamięci Static Random-Access Memory (SRAM) do przechowywania rejonów `.data`, `.bss` i stosu oraz 128 KiB pamięci FLASH, z czego pierwsza połowa jest przeznaczona na rejon `.text` a druga służy jako nieulotne zaplecze pamięci urządzenia MSC.

Aby szybko reagować na zdarzenia magistrali USB, korzystam z wbudowanego Programmable Fast Interrupt Controller (PFIC) bazującego na Vector Table Free (VTF) pozwalający na szybkie wejście do Interrupt Service Routine (ISR) ze wspieranym sprzętowo powrotem do pętli głównej programu. Działanie oparte na przerwaniach ma przewagę względem ciągłego sondowania stanu kontrolera (*polling*), ponieważ pozwala na wykonywanie innych operacji w trakcie działania programu, przyspiesza czas reakcji, co jest ważne w przypadku protokołów komunikacyjnych, gdyż w wielu fazach komunikacji USB są ustalone twarde limity czasowe na odpowiedź. Kolejnym atutem tego podejścia jest dodatkowa informacja płynąca z kontrolera USB – *dlaczego* przerwanie zostało wygenerowane.

2.2. Kontroler USB

Do interakcji z magistralą USB używałem modułu USB Full-speed Device Controller (USBD), który obsługuje komunikację zgodną ze specyfikacją USB 2.0 w trybie Full-speed. Sprzętowo realizuje on kodowanie/dekodowanie Non-Return-to-Zero, Inverted (NRZI) – sposób kodowania sygnału, który rozróżnia bity danych na podstawie obecności lub braku zmiany stanu na krawędziach sygnału zegara, *bit stuffing* – wstawianie bitów nieinformacyjnych do danych w celu odróżnienia danych od informacji sterujących, obsługę pakietów USB (opisanych w 3.4.2.) oraz podstawowe mechanizmy kontroli przepływu. W praktycznej implementacji wykorzystałem mechanizmy z dokumentacji technicznej płytki [15, rozdział 23], jednak bez posługiwania się funkcjami hosta ani dodatku On-The-Go (OTG).

2.3. Inne peryferia

Zapewnienie poprawnego działania pozostałej części urządzenia MSC wymagało również użycia innych peryferiów dostępnych na mikrokontrolerze, podczas gdy kontroler USB pozwalał na komunikację z hostem. Następujące peryferia umożliwiały zachowanie spójnego zachowania wewnątrz urządzenia oraz ułatwiały rozwój:

Kontroler FLASH

Pamięć FLASH jest mapowana pod adres 0x08000000 dzięki czemu odczyty z niej mogą być realizowane przez dostępy z przesunięciem od tego adresu. Choć przez naturę działania pamięci FLASH – wymóg usunięcia danych ze strony (ciągłego bloku pamięci), zanim zostaną tam zapisane nowe dane, sprawił, że zapisy muszą być wykonywane za pomocą wbudowanego kontrolera pamięci FLASH. Obsługuje on tryb *Fast* idealnie pasujący do zapisywania w stylu Logical Block Addressing (LBA), gdyż usuwa/programuje on jedną stronę wielkości 256B na operację, natomiast rozmiar bloku urządzenia MSC wynosi 512B, co pozwala na wykonanie jedynie dwóch operacji usunięcia i zaprogramowania danych na zapis jednego bloku.

RCC

Moduł Reset and Clock Control (RCC) pełni rolę synchronizacji i utrzymania parametrów czasowych między procesorem i peryferiami. Ustala zegar użyty do taktowania procesora, jego częstotliwość, oraz włącza sygnały zegarowe jedynie do potrzebnych peryferiów. W ten sposób nieużywane peryferia nie wysyłają zbędnych sygnałów ani nie zużywają prądu, a peryferia zależne od sygnałów zegarowych (takie jak kontroler USB lub USART) mają dostęp do zegarów odpowiednich częstotliwości.

USART

Universal Synchronous and Asynchronous Receiver-Transmitter (USART) w trybie synchronicznym służył jako główny sposób do diagnozy błędów podczas rozwoju oraz proste rozwiązanie na analizę zachowania samego urządzenia, oraz weryfikację zachowania hosta, aby lepiej zrozumieć sterowniki USB. Wyprowadzenie sygnału USART było realizowane przez jeden z pinów General-Purpose Input/Output (GPIO), na który został zmapowany ten strumień. Następnie podłączyłem go do modułu komunikacyjnego USB na USART, co pozwoliło na naturalne odczytywanie komunikatów na komputerze, np. za pomocą takich programów jak `screen`, opisanego w 6.1.2..

Timer

Timer również został użyty do diagnozy, za pomocą przerwań generował on sygnał o niskiej częstotliwości, który w połączeniu z modulem USART wyzwał „heartbeat” – informację działania urządzenia, co było zwłaszcza przydatne przy operowaniu z pamięcią FLASH, w której znajduje się również kod programu.

Rozdział 3.

Protokoły komunikacyjne

W tym rozdziale przedstawię niektóre z przewodowych protokołów komunikacyjnych powszechnie używanych w projektach wbudowanych. Termin protokół będzie stosowany jako pojęcie **nadrzędne**, obejmujące zarówno warstwę sprzętową, jak i logiczną.

Protokoły komunikacyjne to sposób usystematyzowania technik przekazywania informacji między urządzeniami oraz ujednolicenia komponentów odpowiedzialnych za przekazywanie danych, obejmujących część fizyczną – interfejs (*np. medium transmisyjne*) oraz logiczną – właściwy protokół (*np. topologia, synchronizacja*). Dzięki tej unifikacji różne urządzenia implementujące ten sam standard komunikacyjny mogą płynnie ze sobą współpracować. W zależności od celu komunikacji, środowiska pracy, a także charakterystyki urządzenia końcowego, wybór odpowiedniego protokołu do zadania wymaga znajomości ich właściwości.

3.1. USART

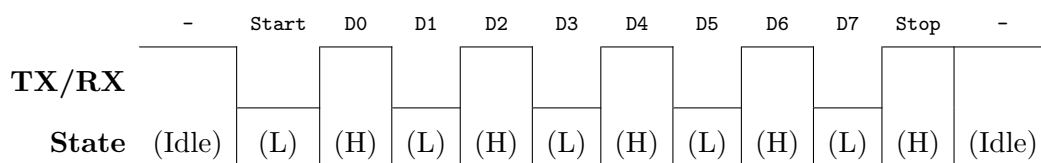
Universal Synchronous and Asynchronous Receiver-Transmitter (USART) [12] jest najprostszym i najwolniejszym powszechnie używanym protokołem, do komunikacji jednostronnej wymaga jedynie trzech linii: **Transmit / Transmitter (TX)**, **Receive / Receiver (RX)**, **Ground (GND)** (*wspólny punkt odniesienia o potencjale 0 V*) i, w jednej z wersji, **Clock (CLK)**. Chociaż jeśli komunikacja ma się odbywać w jedną stronę, można pominąć linię TX lub RX. W kolejnych protokołach, chyba że jest zaznaczone inaczej, GND występuje domyślnie. Ten protokół jest używany głównie jako prosty do konfiguracji kanał diagnostyczny.

Termin USART obejmuje dwa tryby pracy:

- Synchroniczną – w nim występuje linia CLK, przez co transmisja jest synchronizowana zboczem sygnału zegarowego znanego wszystkim podłączonym urządzeniom, chociaż potrzebny jest jeden zegar ustalający tę synchronizację.

- Asynchroniczną – skracany do *UART*, w tym trybie linia CLK nie występuje, przez co podłączone urządzenia muszą z góry ustalić taktowanie, tu znane jako *baud rate*.

Pomimo prostoty, operacje są *full duplex* – przesył (linia TX) oraz odbiór (linia RX) danych mogą zachodzić jednocześnie. Komunikacja asynchroniczna przebiega w ramach o ustalonym rozmiarze, zwykle 8 bitów. W stanie bezczynności linia danych jest utrzymywana w stanie wysokim (H), a transmisja zaczyna się od wygenerowania przez nadajnik *bitu startu*, czyli wymuszeniu stanu niskiego (L) na linii przez czas trwania jednego bitu, ten sygnał synchronizuje zegar odbiornika. Po synchronizacji następuje transmisja z góry ustalonej liczby bitów (7-9) w kolejności od najmniej do najbardziej znaczących. Transmisja kończy się wysłaniem *bitu stopu*, czyli podniesieniem linii do stanu wysokiego na czas trwania co najmniej jednego bitu. Po tym magistrala przechodzi w stan bezczynności i może rozpocząć się kolejna transmisja. Na rysunku 3.1 jest przedstawiona przykładowa ramka danych.



Rysunek 3.1: Przebieg czasowy ramki danych USART podczas transmisji znaku 'U' (0x55).

3.2. SPI i I²C

SPI

Protokół Serial Peripheral Interface (SPI) [22] to najprostszy protokół z topologią, co pozwala na bardziej skomplikowane układy i przekaz informacji niż USART. W omawianych protokołach z topologią pewna jednostka nadrzędna (*Master*) zarządza komunikacją z jednym lub wieloma urządzeniami podrzędnymi (*Slaves*). Głównie użytkowany do prowadzenia komunikacji *full-duplex* mikrokontrolerów z peryferiami. Wymaga co najmniej czterech linii, z możliwością rozszerzenia na więcej linii $\overline{SS}$ aby połączyć więcej Slave'ów, wszystkie linie są opisane w tabeli 3.1.

Kolejną przewagą nad USART'em jest większa swoboda – *teoretycznie* dowolny rozmiar słowa oraz dostosowywanie kolejności bitów, oraz polaryzacji i fazy zegara, czyli w którym dokładnie momencie bity są wystawiane na linię, a w którym odczytywane, co może różnić się w zależności od urządzenia.

Tabela 3.1: Opis i funkcje linii sygnałowych interfejsu SPI

Nazwa linii	Opis	Funkcja
MOSI	<i>Master Output Slave Input</i>	Dane od Mastera do Slave'a
MISO	<i>Master Input Slave Output</i>	Dane od Slave'a do Mastera
SCK	<i>Serial Clock</i>	Sygnał zegarowy
$\overline{SS}$	<i>Slave Select</i>	Linia wyboru Slave'a, zanegowana

I²C

Inter-Integrated Circuit (I²C) [17] jest protokołem podobnym do SPI, lecz charakteryzującym się prostotą budowy i ekonomicznością. Używa on jedynie linii **Serial Data (SDA)** i **Serial Clock (SCL)**, ale również jest w stanie łączyć wiele osobno urządzeń ze sobą. Realizuje to przez bardziej złożoną część logiczną protokołu.

Protokół magistrali zapewnia funkcje niezbędne do poprawnej wymiany danych, takie jak: adresowanie, arbitraż połączony z detekcją kolizji (*collision detection*), odpowiedzi na transmisję w postaci sygnałów Acknowledgement (ACK) i Negative Acknowledgement (NAK) oraz mechanizmy kontroli przepływu. Co warto podkreślić, analogiczne koncepcje potwierdzeń pakietów występują w protokołach CAN 3.3. i USB 3.4., w drugim stanowią one bazę części *handshake* komunikacji.

3.3. CAN

Controller Area Network (CAN) [21] stawia z kolei na niezawodność i odporność na usterki, z tego powodu jest powszechnie używany w branży samochodowej oraz innych środowiskach z zakłóceniami elektromagnetycznymi. Dane są transmitowane za pomocą dwuprzewodowej magistrali różnicowej, co oznacza, że znaczenie ma różnica napięcia między przewodami, może to być implementowane inaczej niż linia danych i **GND**, który domyślnie tu nie występuje. Charakterystyka magistrali różnicowej naturalnie wprowadza pojęcia stanu dominującego (reprezentującego logiczne 0) i recesywnego (logiczną 1). W praktyce oznacza to, że węzeł ściągający linię do stanu dominującego nadpisuje sygnał każdego innego węzła, który w tym samym momencie próbuje nadawać logiczną 1. Przez brak taktowania, wszystkie urządzenia na danej magistrali muszą pracować z tą samą prędkością. Dzięki temu protokół może operować na topologii *Multimaster*, czyli nie ma wyznaczonego jednego sztywno ustalonego Mastera, za to każde z urządzeń ma pewien priorytet. Jeżeli dwa urządzenia rozpoczną transmisję w tym samym momencie, następuje nie-destrukcyjny arbitraż i zwycięża urządzenie z większym priorytetem.

Protokół CAN zawiera wiele sposobów na wykrywanie, sygnalizowanie i naprawianie błędów powstałych na magistrali, ze względu na strukturę sieci każdy węzeł

ma równe prawo do monitorowania jej stanu. W przypadku wykrycia nieprawidłowości może oznakować wiadomość jako uszkodzoną oraz monitorować błędy w swoim otoczeniu, chociaż istnieją też zasady w przypadkach, gdy błąd występuje w węźle.

3.4. USB

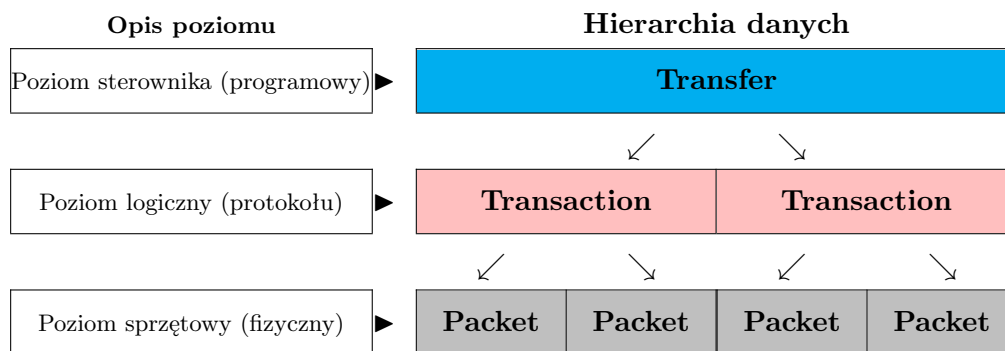
Ostatnim omawianym protokołem i tym, na którym się skupię, będzie Universal Serial Bus (USB) w wersji 2.0 i prędkości *Full Speed*. Został zaprojektowany jako *uniwersalny* protokół do wielu przeznaczeń jednym kablem, komputer potrzebuje jedynie załadować odpowiedni sterownik dla urządzenia po jego rozpoznaniu. Podobnie jak CAN używa on różnicowego przesyłu danych, jednak posiada dwie dedykowane linie dla danych **D+** i **D-** oraz dwie linie zasilania **VCC** (zasilanie) i **GND**. Dzięki takiej strukturze realizuje on zasilanie urządzeń oraz komunikację z nimi bez potrzeby prowadzenia zasilania osobno.

3.4.1. Topologia

USB operuje na topologii ścisłej gwiazdy piętrowej, jeden nadrzędny *host* komunikuje się z urządzeniami podłączonymi bezpośrednio lub do węzłów komunikacyjnych (*Hubs*) połączonych w strukturze drzewiastej. *Host* posiada całkowitą kontrolę nad urządzeniami pod nim przez zarządzanie ich zasilaniem i samą magistralą. Transmisja danych realizowana jest przez odpytywanie urządzeń (*Polling*), co oznacza, że urządzenie peryferyjne może rozpocząć nadawanie wyłącznie w odpowiedzi na bezpośrednie żądanie wystosowane przez *Hosta*.

3.4.2. Przesył danych

Sposób przesyłania danych w protokole USB jest standaryzowany przez **transfery**. Są to pełne operacje komunikacyjne z wysokopoziomowego punktu widzenia (np. sterownika), które wykonują w całości ustalone zadanie takie jak przesłanie pliku lub ustawienie konfiguracji. Transfery są podzielone na **transakcje**, są to podstawowe, niepodzielne jednostki ruchu USB składające się z **pakietów**. Jeśli jakaś część transakcji się nie powiedzie, cała transakcja musi zostać powtórzona, jest to realizowane całkowicie przez sterownik sprzętowy. Na rysunku 3.2 jest przedstawiona struktura transmisji danych w USB.



Rysunek 3.2: Podział i hierarchia danych w transmisji USB.

Pakiety

Pakiety to bloki bazowe budujące transakcje [20, rozdział 3]:

1. **Pakiety Token** - informuje urządzenie, jaki rodzaj transakcji rozpoczął *host* spośród: **In** (*host* chce czytać dane), **Out** (*host* chce wysłać dane) i **Setup** (rozpoczynanie transferów kontrolnych, opisanych w 4.2.).
2. **Pakiety Data** - wysyłane naprzemiennie jako **Data0** i **Data1** dla zabezpieczenia przed zagubieniem pakietów, używane do przesyłania właściwych danych (*payload*) o rozmiarze maksymalnie 64 bajtów dla transferów masowych (*Bulk*), omawianych w 4.3..
3. **Pakiety Handshake** - odpowiedzi na inne pakiety spośród: **ACK** (pomyślnego odbioru pakietu), **NAK** (tymczasowy brak możliwości wysyłania lub odbierania danych oraz specyficzne sytuacje w protokole) i **STALL** (urządzenie jest w niepoprawnym stanie i wymaga interwencji hosta).
4. **Pakiety Start of Frame** - znaczniki czasu wysyłane co $1\text{ ms} \pm 500\text{ ns}$ do synchronizacji i ustalenia przedziałów pakietów.

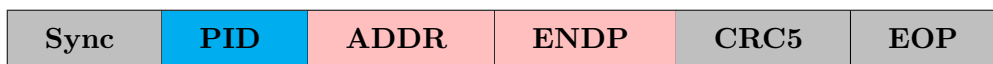
Transakcje biorące swe przeznaczenie od *Packet* lub typu pakietu **Token** będą skracane zapisem transakcja *Packet/Token*.

Pola pakietów

Pakiety są kombinacjami tych pól [20, rozdział 3]:

- **Sync** – 8-bitowe pole, którym zaczynają się wszystkie pakiety, używany do synchronizacji zegarów hosta i urządzenia oraz wskazania pozycji następnego wymienionego pola w pakiecie.

- **PID** – 4-bitowe pole *Packet ID*, jest powtarzane raz, aby zapewnić poprawny odbiór, oznacza rodzaj wysyłanego pakietu.
- **ADDR** – 7-bitowe pole adresu docelowego pakietu, adres 0 to adres ogłoszeniowy dla urządzeń bez adresu.
- **ENDP** – 4-bitowe pole docelowego endpointu (opisanego w 3.4.2.).
- **CRC** – 5/16-bitowe pole zawiera cykliczny kod nadmiarowy, który służy do weryfikacji integralności przesyłanych danych przez odbiorcę.
- **EOP** – Pole kończące pakiet, jest wysyłane przez wymuszenie obu stanów linii danych na niski przez 2 bity, a następnie powrót do stanu bezczynnego na 1 bit.



Rysunek 3.3: Struktura pól pakietu Token w protokole USB.

3.4.3. Punkty końcowe

Punkt końcowy (*endpoint*) można opisać jako pojedyncze adresowalne źródło lub ujście danych w strukturze samego urządzenia [20, rozdziały 3,4]. Operuje jako logiczne bloki z przypisanymi kierunkami i typami transferów, każde urządzenie składa się ze zbioru niezależnych *endpointów*. Ze względu na szeregowy charakter transmisji, służą one jako bufor danych w kontrolerze urządzenia. Pozwala to na asynchroniczne przygotowywanie pakietów do wysłania (bufory typu IN) oraz zachowanie otrzymanych danych do przetworzenia (bufory typu OUT). Terminy IN i OUT zawsze odnoszą się do kierunku transferu **z perspektywy hosta**.

Rozdział 4.

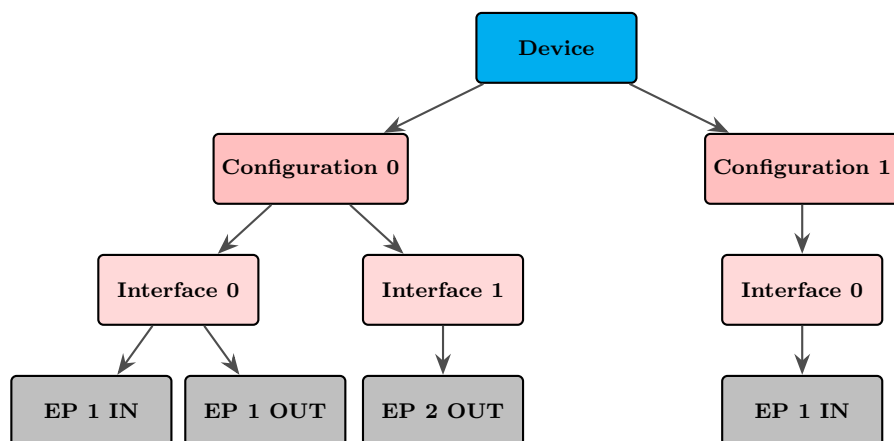
Struktury i transfery danych USB

Aby zrealizować postawione zadanie zaimplementowania urządzenia Mass Storage Class (MSC) [2] wymagane było zrozumienie struktur danych zdefiniowanych w specyfikacji, ich hierarchię oraz obsługa dwóch rodzajów transferów: **kontrolnych**, oraz **masowych**. Do obsługi transferów kontrolnych domyślnie jest używany Endpoint (EP) 0, taki standard został wybrany, ponieważ urządzenie na magistrali musi przed rozpoczęciem działania zostać zidentyfikowane przez *hosta* w procesie zwanym *enumeracją*, szczegółowo opisanym w sekcji 4.2.. Transfery drugiego typu realizowałem za pomocą pary EP 1 IN oraz EP 1 OUT, pomimo dzielenia indeksu są to dwa logiczne punkty końcowe.

4.1. Deskryptory i struktura logiczna

Specyfikacja USB [26, sekcja 9.5] wprowadza terminy deskryptor (*descriptor*), interfejs (*interface*) i konfiguracja (*configuration*), aby opisywać własności i zachowania urządzeń, transfery kontrolne pozwalają odpytywać urządzenia o te struktury danych i nimi zarządzać. Deskryptor to struktura danych o ściśle ustalonym formacie na potrzeby opisanego pól w hierarchii atrybutów urządzenia. Całość przyjmuje strukturę drzewiastą, korzeniem jest zawsze *device descriptor* 4.1., który odnosi się do topologii ustawień urządzenia, przykładowa struktura jest przedstawiona w rysunku 4.1.

Ten model został opracowany, aby urządzenia z konfiguracjami dzielącymi części charakterystyk mogły ponownie wykorzystywać te same deskryptory. W ten sposób deskryptory przypominają rekordy w relacyjnych bazach danych.



Rysunek 4.1: Przykładowa struktura deskryptorów urządzenia USB.

Device descriptor

Device descriptor służy do opisu ogólnych informacji urządzenia USB, jest ko-
rzeniem hierarchii opisowej urządzenia. Zawiera krytyczne informacje obowiązujące
przy każdej komunikacji z tym urządzeniem: numer wydania specyfikacji USB, z
którą urządzenie jest zgodne, kod klasy USB i jej protokołu, jeśli urządzenie operuje
na protokole poziomu urządzenia, maksymalny rozmiar pakietu dla domyślnego kon-
trolnego EP 0 (przed przesłaniem tego deskryptora host nie wie, jak dzielić pakiety),
informacje o pochodzeniu urządzenia i ilość konfiguracji.

Configuration descriptor

Configuration descriptor opisuje konkretną konfigurację urządzenia, które może
implementować wiele konfiguracji, jednak tylko jedna z nich jest wybrana w danym
momencie. Obejmuje głównie właściwości zasilania urządzenia oraz ilość interfejsów.

Interface descriptor

Interface descriptor określa specyficzny tryb pracy (interfejs) urządzenia w kon-
figuracji, dopuszczającej wiele interfejsów. Poza opisem ilości *endpointów* interfejs
może dostarczać alternatywne ustawienie – inne *endpointy* lub ich uregulowanie. W
tym deskrypcorze występuje również kod klasy i jej protokołu, na poziomie interfejsu.
Klasa MSC jest wybierana na tym etapie, razem z komendami SCSI 5..

Endpoint descriptor

Na dole hierarchii znajduje się *Endpoint descriptor*, który jednoznacznie defi-
niuje parametry punktu końcowego. Poza identyfikacją unikalnego numeru, maksy-
malnego obsługiwanego rozmiaru pakietu i kierunku dostarcza informacji o obsłu-

giwanym typie transferów, wymaganym interwale odpytywania magistrali o dane, dopuszczalnej częstotliwości odpytywania po odpowiedzi NAK (dla *endpointów* obsługujących transfery masowe) oraz parametrów fizycznych. Standardowy *endpoint* 0 nie może zostać opisany deskryptorem, ma on sztywno ustalone wymagania w specyfikacji [26, podsekcja 4.8.1]. EP 1 IN oraz OUT są skonfigurowane jako *endpointy* transferu *Bulk-Only* 4.3. z maksymalnym rozmiarem pakietu 64 bajtów.

Warto zaznaczyć, że implementacja zaplecza pamięci zawsze kończy dostępy do pamięci przed odpowiedzią, więc w trakcie transferu sytuacja odpytywania o postęp przetwarzania pakietu się nie zdarzy.

4.2. Transfery kontrolne i potok sterujący

Transfery kontrolne to niezbędny zestaw operacji umożliwiający *hostowi* konfigurację, zarządzanie i sprawdzanie statusu różnych części urządzenia. Transfer kontrolny zaczyna się od transakcji *Setup* 3.4.2. wskazującej cel i kierunek następnych transakcji, następnie wysyłanych jest zero lub więcej transakcji *Data*, a ostatecznie transakcja *Status* (której głównym elementem jest *Handshake*), aby upewnić się, że *endpoint* zakończył przetwarzać żądane dane.

Komunikacja *hosta* z urządzeniem rozpoczyna się od zmiany stanu jednej z linii danych przez urządzenie ze stanu spoczynkowego, sygnalizując obecność i jaki jest tryb prędkości – w przypadku wybranego standardu *Full-speed* jest to podciągnięcie linii **D+** do stanu wysokiego. Następnie rozpoczyna się enumeracja urządzenia, czyli proces wykrywania, identyfikacji i ładowania sterowników dla urządzenia USB [5] [26, podsekcja 9.1.2].

Enumeracja

Opisy wszystkich używanych deskryptorów znajdują się w sekcji **Deskryptory i struktura logiczna 4.1.**

1. **Wykrycie i reset:** *Host* upewnia się, że zmiana na linii **D+** nie jest przypadkowa, jeśli jest stabilna, to wysyła do urządzenia sygnał resetu, po tym urządzenie musi odpowiadać na transfery kontrolne pod adresem 0, z ograniczonym zasilaniem.
2. **Wstępne pobranie parametrów (opcjonalne):** W praktycznych implementacjach systemów operacyjnych, *host* przesyła żądanie `GET_DESCRIPTOR` typu `DEVICE` na adres 0 w celu odczytania pierwszych 8 bajtów deskryptora urządzenia i poznania maksymalnego rozmiaru pakietu dla EP 0, następnie urządzenie jest ponownie resetowane.

3. **Ustawienie adresu:** *Host* nadaje urządzeniu unikalny adres, na który ma reagować.
4. **Uzyskiwanie deskryptorów:** Za pomocą komend `GET_DESCRIPTOR` *host* uzyskuje pełne deskryptory urządzenia, konfiguracji, interfejsów i punktów końcowych.
5. **Załadowanie sterownika:** Po pełnej identyfikacji urządzenia *host* przydziela odpowiedni sterownik do zarządzania urządzeniem, w przypadku urządzenia MSC jest to sterownik pamięci masowej.
6. **Ustawienie konfiguracji:** Komendą `SET_CONFIGURATION` *host* wysyła żądanie aktywacji wybranej konfiguracji (opcjonalnie alternatywnych interfejsów), po niej oczekuje, że urządzenie jest skonfigurowane i punkty końcowe EP 1 IN i EP 1 OUT są gotowe do transferów masowych.

4.3. Transfery masowe i protokół Bulk-Only Transport

Transfery masowe (*Bulk*) są używane do dużych, sporadycznych przesyłów danych np. otrzymywanie obrazów ze skanera albo przesyłanie danych na dysk zewnętrzny. Zapewniają mocniejsze wykrywanie błędów w postaci 16-bitowego pola **CRC 3.4.2.** i dodatkowe mechanizmy wykrywania błędów i retransmisji zapewniające, że dane nie zostaną uszkodzone przy przesyśle. EP 1 IN oraz EP 1 OUT to jedyne i unikalne punkty końcowe typu *Bulk*, więc numeracja przy nich zostanie opuszczona.

Host odpowiada za zarządzanie przepustowością łącza magistrali, specyfikacja plasuje transfery masowe jako najmniej krytyczne, dlatego przepustowość jest im przydzielana po wszystkich innych rodzajach transferów. Z tego powodu te transfery powinny być używane jedynie do komunikacji bez nałożonych ram czasowych [20, rozdział 4].

4.3.1. Architektura protokołu Bulk-Only Transport

Sposób wykorzystania transferów masowych do realizacji poleceń, przesyłania danych oraz statusów definiuje protokół Bulk-Only Transport (BOT) [25]. Eliminuje potrzebę stosowania transferów kontrolnych do obsługi bieżących operacji, komunikacja odbywa się wyłącznie za pomocą punktów końcowych typu *Bulk*, poza obowiązkowym EP 0.

Struktury danych

BOT jest oparty o dualne struktury danych Command Block Wrapper (CBW) i Command Status Wrapper (CSW) ze ściśle ustalonymi wyrównaniami i rozmiarami

w celu uodpornienia tego rodzaju transmisji na uszkodzenie danych.

Command Block Wrapper (CBW) jest wysyłany od *hosta* do urządzenia, ma dokładnie 31 bajtów. Wprowadza ona unikalny format opakowania, wewnątrz którego umieszczany jest właściwy blok polecenia. W przypadku mojego projektu jest to komenda standardu SCSI, omawianego w 5.. Najważniejsze pola tej struktury to:

- **dCBWSignature** (4 bajty) – podpis pomagający zidentyfikować pakiet danych jako CBW, przyjmuje wartość 0x43425355.
- **dCBWTag** (4 bajty) – etykieta *Command Block* nadawana przez *hosta* która musi zostać odesłana w CSW będącym odpowiedzią na ten CBW. Bezwzględnie identyfikuje odpowiedź z konkretnym żądaniem.
- **dCBWDataTransferLength** (4 bajty) – liczba bajtów danych, jakich *host* oczekuje do przesłania w **fazie danych** 4.3.1., może wynosić 0, wtedy faza danych jest pomijana.
- **bmCBWFlags** (1 bajt) – kierunek transferu danych.
- **CBWCB** (1-16 bajtów) – skrót od *Command Block Wrapper Command Block*. Jest to właściwy blok komendy zawierający wysokopoziomowe komendy standardu SCSI.

Po przetworzeniu żądania i zakończeniu fazy danych urządzenie odpowiada przy pomocy struktury Command Status Wrapper (CSW) o stałej długości 13 bajtów:

- **dCSWSignature** (4 bajty) – podpis pomagający zidentyfikować pakiet danych jako CSW, jego wartość to 0x53425355.
- **dCSWTag** (4 bajty) – etykieta odpowiadająca CBW, do którego odpowiedzią jest ten CSW.
- **dCSWDataResidue** (4 bajty) – różnica między ilością danych oczekiwanych, zgodnie z wartością **dCBWDataTransferLength**, a rzeczywistą ilością danych przetworzonych przez urządzenie.
- **bCSWStatus** (1 bajt) – wskazuje na powodzenie lub niepowodzenie polecenia.

Żądania

Protokół BOT definiuje dwa dodatkowe żądania względem urządzenia. Pierwszym z nich jest **Bulk-Only Mass Storage Reset** polegający na zresetowaniu urządzenia MSC wraz z powiązanim z nim interfejsem, jest wysyłany na EP 0. Drugim jest **Get Max LUN** odpytujący urządzenie o liczbę implementowanych jednostek logicznych (np. mediów danych) dzielących charakterystyki urządzenia. Urządzenie implementowane przeze mnie nie zawiera takich jednostek.

Fazy

Bulk-Only Transport (BOT) jest podzielony na trzy fazy, transport jest uznany za pomyślny, jeżeli wszystkie są wykonane sekwencyjnie i bezbłędnie:

1. **Faza komendy** (*Command Transport*) - host wysyła CBW do urządzenia za pomocą EP OUT.
2. **Faza danych** (*Data-In/Data-Out*) - przesył właściwych danych w kierunku w ilości i kierunku zależnym od wartości w poprzedzającym CBW (przez EP IN lub EP OUT)
3. **Faza statusu** (*Status Transport*) - urządzenie wysyła CSW do hosta za pośrednictwem EP IN. CSW informuje hosta o stanie wykonania polecenia.

Potrzeba niezawodności powoduje, że BOT zapewnia zaawansowany wachlarz wykrywania i obsługi błędów. W celu obsługi sytuacji wyjątkowych poza błędami CBW i CSW specyfikacja BOT definiuje tzw. *The Thirteen Cases* [25, sekcja 6.7] – macierz scenariuszy określających zachowanie stron w przypadku niespójności danych, dogłębnie definiuje poprawne reakcje protokołu na rozbieżności pomiędzy oczekiwaniami *hosta* a intencjami urządzenia.

Rozdział 5.

Komendy SCSI

Podczas gdy protokół Bulk-Only Transport (BOT) odpowiada wyłącznie za danych na niższym poziomie, tym, co nadaje im sens komend urządzenia blokowego jest Small Computer System Interface (SCSI). Komendy znajdują się w polu CBWCB struktury CBW i ściśle współgrają z fazą danych protokołu transportowego.

Rodzina standardów SCSI definiuje mechanizmy i operacje wymagane do zapewnienia współpracy pomiędzy zgodnymi implementacjami SCSI. Kluczową zaletą tej architektury jest całkowita abstrakcja warstwy sprzętowej – standard ten definiuje wysokopoziomowy, logiczny interfejs wymiany informacji, co pozwala systemom operacyjnym na implementację generycznych sterowników. Na tym poziomie system traktuje podłączone urządzenie jako standaryzowaną pamięć blokową, niezależną od części fizycznej.

W architekturze SCSI komunikacja jest podobna do relacji *host-device* w USB. *Host* wysyła żądania, natomiast *device* jest odpowiedzialne za interpretację odebranych struktur i realizację niskopoziomowych operacji sprzętowych na własną rękę.

Aby zrealizować postawiony cel, jakim jest emulacja funkcjonalnego urządzenia pamięci masowej, potrzebny był krytyczny podzbiór komend z trzech standardów: **SPC-3**, **SBC-2** i **SSC-2**.

5.1. SPC-3

Specyfikacja SCSI Primary Commands - 3 (SPC-3) [9] określa obowiązkowe i opcjonalne komendy SCSI dla wszystkich urządzeń implementujących standard SCSI, stanowi fundament dla innych standardów definiujących komendy i zachowania dla wyspecjalizowanych urządzeń. Obsługa większości komend opisanych w tym standardzie jest dobrowolna lub odnosi się do innych rodzajów urządzeń, więc systemowy sterownik ich nie użyje względem prostej implementacji urządzenia MSC jak w tym projekcie.

5.1.1. Komendy specyfikacji SPC-3

W ramach tego standardu była niezbędna obsługa sześciu kluczowych komend odpowiedzialnych za identyfikację urządzenia względem SCSI, diagnostykę i proste zarządzanie nośnikiem. Wszystkie komendy w używanych standardach traktują pierwszy bajt jako kod operacji, będzie on podawany wraz z nazwą komendy. Liczba w nawiasach po nazwie komendy na przykład **READ (10)** oznacza wariant rozmiaru komendy w bajtach. Wszystkie odpowiedzi na komendy są ściśle strukturyzowane, przy ich opisywaniu zaznaczam jedynie najważniejsze pola.

INQUIRY 0x12

Komenda wysyłana na samym początku komunikacji SCSI służy do pobrania podstawowych parametrów o urządzeniu oraz opcjonalnie dodatkowych istotnych danych, końcowo określa maksymalny rozmiar odpowiedzi od urządzenia.

W odpowiedzi urządzenie identyfikuje, z jakim standardem jest zgodne i jakiego typu peryferium jest (w tym przypadku *Direct Access Block Device*) oraz nazwy producenta i modelu urządzenia w postaci ciągów *ASCII*.

TEST UNIT READY 0x00

Często używana komenda wysyłana cyklicznie przez sterowniki. Przeznaczona do odpytania urządzenia, czy jest obecnie w stanie wykonywać operacje dostępu do medium bez zwracania niepoprawnego statusu.

Jeśli urządzenie jest gotowe powinno od razu zwrócić CSW ze statusem sukcesu, w przeciwnym przypadku zgłasza błąd, a *host* powinien wysłać **REQUEST SENSE** 5.1.1. jako następną komendę, aby dowiedzieć się więcej o naturze problemu.

MODE SENSE (6) 0x1A

Komenda służy do pobierania od urządzenia informacji dotyczących jego aktualnej konfiguracji, parametrach pracy oraz właściwościach nośnika. W przypadku prostych urządzeń pamięci masowej system operacyjny używa tej komendy do zwerifikowania dwóch kluczowych parametrów: czy nośnik jest zabezpieczony przed zapisem (*Write Protect bit*) oraz jakie są globalne parametry buforowania.

W odpowiedzi urządzenie odsyła strukturyzowany nagłówek (oraz opcjonalnie bloki danych stron) zawierający typ nośnika, ochronę zapisu i długość deskryptorów bloków po nagłówku, są to struktury danych, które definiują fizyczną lub logiczną geometrię nośnika danych. W moim przypadku nie przesyłam deskryptorów bloków

– korzystam z dedykowanej, obowiązkowej komendy ze standardu SBC-2: **READ CAPACITY (10)** 5.2.1..

PREVENT ALLOW MEDIUM REMOVAL 0x1E

Polecenie to służy do zablokowania lub odblokowania możliwości fizycznego wysunięcia nośnika z napędu przez urządzenie. W kontekście urządzeń MSC, komenda ta jest wysyłana przez system operacyjny bezpośrednio przed rozpoczęciem operacji zapisu lub odczytu, aby zapobiec odłączeniu urządzenia w trakcie krytycznych operacji na systemie plików.

Emulowane urządzenie nie posiada fizycznego medium, które może odłączyć, więc natychmiastowo zwraca status powodzenia.

REQUEST SENSE 0x03

To najbardziej zaawansowana do obsługi komenda tego standardu jest wysyłana przez *hosta* gdy urządzenie zwróci błąd w poprzedniej komendzie. Specyfikacja SPC-3 wymaga od urządzenia przechowywania informacji o błędzie, który wystąpił podczas obsługi komendy do czasu przyjścia następnego polecenia. W skład zwracanych danych diagnostycznych (*Sense Data*) wchodzi dwa kody wyróżniające typ błędu i parametry dodatkowe, wskazujące m.in. winny komponent sprzętowy lub lokalizację nieprawidłowych informacji w komendzie.

Dane diagnostyczne stałego rozmiaru (*Fixed format sense data*) są zwracane przez mikrokontroler wraz z informacjami zapisanymi podczas wykonywania poprzedniej komendy.

5.2. SBC-2

Standard SCSI Block Commands - 2 (SBC-2) [8] definiuje rozszerzenia zestawu poleceń ułatwiające obsługę urządzeń blokowych SCSI z bezpośrednim dostępem. W połączeniu z SPC-3 5.1. stanowi pełny zestaw instrukcji do obsługi urządzeń blokowych. Poniżej przedstawiam opisy komend wymaganych do osiągnięcia pełnej funkcjonalności urządzenia.

5.2.1. Komendy specyfikacji SBC-2

READ CAPACITY (10) 0x25

Ta komenda żąda przesłania danych opisujących pojemność i format nośnika urządzenia blokowego, jest używana przy urządzeniach stosujących Logical Block

Adressing (LBA).

W odpowiedzi urządzenie przesyła numer dostępnego bloku logicznego na nośniku (co pozwala *hostowi* wyliczyć całkowity rozmiar pamięci) oraz rozmiar pojedynczego bloku, który będzie stosowany w LBA, w bajtach.

READ (10) 0x28

Przechodzimy do najważniejszych komend z punktu widzenia działania pamięci masowej, komenda **READ (10)** żąda od urządzenia odczytu pewnej ilości bloków od określonego adresu z uwzględnieniem flag ochrony odczytu i buforowania. Informacje będą przesłane do buforu wejściowego *hosta* pakietami, w ramach fazy danych protokołu transportowego 4.3., CSW zostanie przesłane pod koniec transmisji.

WRITE (10) 0x2A

Drugą z kluczowych operacji jest **WRITE (10)** informuje urządzenie o nadchodzących pakietach danych z buforu wyjściowego *hosta* zawierającym określone flagi ochrony zapisu i buforowania, który ma być zapisany od podanego adresu. W tym przypadku w fazie danych urządzenie jedynie odbiera dane i pod koniec potwierdza pomyślny zapis całego przesłanego zbioru informacji.

START STOP UNIT 0x1B

Ostatnia komenda z SBC-2 umożliwia *hostowi* kontrolowanie zasilania jednostki. Metoda ta obejmuje polecenie załadowania lub wysunięcia nośnika, a także pozwala na aktywację lub dezaktywację urządzenia blokowego poprzez zarządzanie stanami zasilania i wewnętrznymi licznikami (*timers*). Urządzenia z pamięcią podręczną muszą zapisać wszystkie buforowane bloki logiczne na nośniku przed wykonaniem tej operacji, a urządzenia z zegarami mają dodatkowe polecenia.

5.3. SSC-2

SCSI Stream Commands - 2 (SSC-2) [1] nie posłużył jako źródło komend, jako że definiuje on funkcjonalne wymagania urządzeń strumieniowych z sekwencyjnym dostępem (takich jak pamięć taśmowa). Był on potrzebny, ponieważ pola struktur odpowiedzi na komendy w poprzednich standardach (w szczególności **REQUEST SENSE** 5.1.1.) miały pola z referencjami w tym standardzie.

Rozdział 6.

Diagnostyka, testowanie i weryfikacja

W tym rozdziale przedstawię sposób tworzenia oprogramowania będącego obiektem tej pracy oraz narzędzia ułatwiające rozwój i pozwalające na analizę zachowania programu.

6.1. Środowisko deweloperskie i narzędzia

Rozwój projektu, zwłaszcza dla systemów wbudowanych realizujących złożone protokoły komunikacyjne, takie jak USB MSC, wymaga stworzenia dedykowanego środowiska testowego. Opiszę sposób rozwoju, zastosowane programy systemu Linux oraz procedury testowe, które pozwoliły na pełną weryfikację poprawności implementacji.

6.1.1. Środowisko deweloperskie

Projekt korzysta z *toolchainów* `riscv64-elf` do kompilacji, linkowania i budowania projektu. Wszystkie te operacje mogą zostać wykonane za pomocą komendy `make`, samej dla kompilacji i linkowania, z sufiksem `flash` i `lst` dla instalacji oprogramowania na płytkę i *disassembly* odpowiednio. Dodatkowo w repozytorium znajduje się kod producenta **WCH** do obsługi peryferiów oraz konfiguracji specyficznej dla platformy sprzętowej (pakiet CH32V307 EVT) [18] w katalogach `/platform` i `/vendor`, struktura projektu jest dokładniej opisana w repozytorium GitHub [28].

6.1.2. Narzędzia

W celu analizy niskopoziomowej komunikacji oraz zachowania sterowników systemowych Linux wykorzystałem zestaw narzędzi:

Narzędzia diagnostyczne

- **dmesg** [10] – używany z flagą **-w** (śledzenie zmian), kluczowe narzędzie do obserwowania logów jądra systemu (*kernel logs*). Umożliwia odczytywanie reakcji sterownika USB na podłączenie mikrokontrolera i sprawdzenie błędów, kiedy w trakcie kolejnego kroku komunikacji nie działa poprawnie.
- **screen** [4] – wspomniany w sekcji o peryferiach 2.3. emulator terminala. Możliwość wypisywania dowolnych wiadomości pozwala nie tylko na proste przeczytanie pakietów w kluczowych momentach, ale też ich czasów ich nadejścia, to narzędzie zbliżyło proces rozwoju do tworzenia programów natywnych.
- **lsusb** [24] – używany z flagami **-t** (widok drzewa struktury) albo **-v** (tryb szczegółowy), jego celem jest wyświetlanie informacji o magistralach USB w systemie i podłączonych do nich urządzeniach w sposób czytelny dla człowieka. Był użyty do sprawdzania statycznych danych związanych z protokołem USB w szczególności poprawności przesłanych deskryptorów (sekcja 4.1.).
- **lsblk** [14] – program do wyświetlania informacji o urządzeniach blokowych, służył do sprawdzania poprawności rejestracji jako urządzenie blokowe i wykrycia systemu plików.
- **wireshark** [3, 6] – zaawansowany analizator ruchu sieciowego i komunikacyjnego, pozwalał na przechwytywanie surowych pakietów USB celu badania poprawności kolejnych faz komunikacji. Używany głównie do sprawdzania informacji nieraportowanych przez inne narzędzia.

Narzędzia weryfikacyjne

- **dd** [19] – program przeznaczony do niskopoziomowego kopiowania i konwersji surowych plików. Wykorzystany do prostych testów transferu danych.
- **mkfs.fat** [27] – służy do tworzenia systemów plików **FAT** na urządzeniu po pomyślnym zakończeniu rejestracji urządzenia.
- **fsck** [23] – sprawdza i opcjonalnie próbuje naprawiać systemy plików. Pomocny przy wysokopoziomowej weryfikacji poprawności komend zapisu i odczytu.
- **mount** [11] – dołącza emulowany system plików na urządzeniu do systemowego drzewa plików.

6.2. Efekt i testy

Po odpowiedniej implementacji rozdziałów 4–6 można przeanalizować i zweryfikować poprawność działania mikrokontrolera względem planowanych punktów.

Za pomocą przedstawionych narzędzi sprawdziłem reakcję systemowych sterowników Linux na komunikację emulowanego urządzenia MSC oraz wygląd i poprawność jego działania. Testy zostały przeprowadzone w środowisku systemu operacyjnego Manjaro Linux z jądrem w wersji 6.12.77-1-MANJARO.

6.2.1. Zachowanie sterowników

Pierwszym krokiem weryfikacji jest sprawdzenie dzienników (*logs*) jądra 6.1 w celu sprawdzenia sposobu interpretacji nowo podłączonego urządzenia przez *hosta*.

```
usb 3-1: new full-speed USB device number 53 using xhci_hcd
usb 3-1: New USB device found, idVendor=0000, idProduct=0000, bcdDevice= 1.00
usb 3-1: New USB device strings: Mfr=0, Product=0, SerialNumber=0
usb-storage 3-1:1.0: USB Mass Storage device detected
scsi host2: usb-storage 3-1:1.0
scsi 2:0:0:0: Direct-Access      NONE          RISC-V SOC MSC    0.01 PQ: 0 ANSI: 0
sd 2:0:0:0: [sda] 128 512-byte logical blocks: (65.5 kB/64.0 KiB)
sd 2:0:0:0: [sda] Write Protect is off
sd 2:0:0:0: [sda] Mode Sense: 03 00 00 00
sd 2:0:0:0: [sda] No Caching mode page found
sd 2:0:0:0: [sda] Assuming drive cache: write through
sda:
sd 2:0:0:0: [sda] Attached SCSI removable disk
```

Rysunek 6.1: Komunikaty jądra z `dmesg` po podłączeniu urządzenia USB MSC.

Dzienniki pokazują poprawną rejestrację urządzenia jako urządzenie blokowe z komendami SCSI. Kolejnym etapem walidacji jest analiza sposobu wykrywania interfejsów na poziomie sterowników USB, przedstawiona w listingu 6.2.

```
Port 001: Dev 053, If 0, Class=Mass Storage, Driver=usb-storage, 12M
```

Rysunek 6.2: Połączenie interfejsu urządzenia ze sterownikiem `usb-storage` w `lsusb`.

Widać poprawne urządzenie MSC, następnie należy sprawdzić urządzenie blokowe.

NAME	SIZE	START TYPE	FSTYPE
sda	64K	disk	vfat

Rysunek 6.3: Rozpoznanie urządzenia blokowego w `lsblk`.

Ostatnim krokiem jest dokładna analiza przesłanych informacji o urządzeniu w postaci deskryptorów (opisanych w 4.1.) i weryfikacja poprawności struktury drzewiastej, przedstawionej w listingu 6.4.

```

Bus 003 Device 053: ID 0000:0000
Couldn't open device, some information will be missing
Negotiated speed: Full Speed (12Mbps)
Device Descriptor:
  bLength                18
  bDescriptorType        1
  bcdUSB                 2.00
  bDeviceClass            0 [unknown]
  bDeviceSubClass        0 [unknown]
  bDeviceProtocol        0
  bMaxPacketSize0        64
  idVendor               0x0000
  idProduct              0x0000
  bcdDevice              1.00
  iManufacturer          0
  iProduct               0
  iSerial                0
  bNumConfigurations     1
Configuration Descriptor:
  bLength                9
  bDescriptorType        2
  wTotalLength           0x0020
  bNumInterfaces         1
  bConfigurationValue    1
  iConfiguration         0
  bmAttributes           0x80
    (Bus Powered)
  MaxPower               100mA
Interface Descriptor:
  bLength                9
  bDescriptorType        4
  bInterfaceNumber       0
  bAlternateSetting      0
  bNumEndpoints          2
  bInterfaceClass        8 Mass Storage
  bInterfaceSubClass     6 SCSI
  bInterfaceProtocol     80 Bulk-Only
  iInterface             0
Endpoint Descriptor:
  bLength                7
  bDescriptorType        5
  bEndpointAddress       0x01  EP 1 OUT
  bmAttributes           2
    Transfer Type        Bulk
    Synch Type           None
    Usage Type           Data
  wMaxPacketSize         0x0040  1x 64 bytes
  bInterval              0
Endpoint Descriptor:
  bLength                7
  bDescriptorType        5
  bEndpointAddress       0x81  EP 1 IN
  bmAttributes           2
    Transfer Type        Bulk
    Synch Type           None
    Usage Type           Data
  wMaxPacketSize         0x0040  1x 64 bytes
  bInterval              0

```

Rysunek 6.4: Szczegółowa struktura deskryptorów urządzenia w `lsusb -v`.

Wszystkie przesłane informacje zostały poprawnie odebrane.

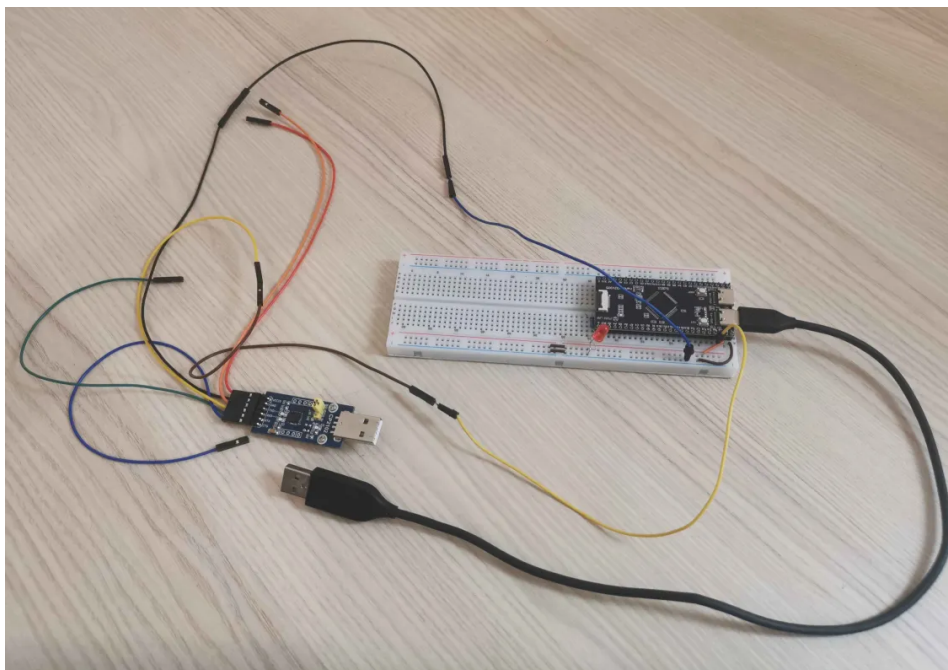
6.2.2. Testy funkcjonalne

Końcowym etapem weryfikacji jest test rzeczywistego użytkowania urządzenia. W tym celu przygotowałem skrypt `test.sh`, za jego pomocą można wykonać pełny test: wyczyszczenie urządzenia, stworzenie systemu plików, sprawdzenie jego poprawności, zamontowanie go, przesłanie pliku, a następnie pobranie go i sprawdzenie poprawności zapisu oraz odczytu.

6.2.3. Efekt

Ostatecznym efektem urządzenie przedstawia się jako nowy, wymienny dysk operujący na komendach SCSI, można do niego manualnie przesyłać dane za pomocą narzędzi takich jak `dd`. Aby w pełni osiągnąć emulację urządzenia typu *pen-drive* potrzebne jest stworzenie systemu plików (przetestowane z `mkfs.fat` tworzącym system **FAT12**) na nośniku i zamontowanie go. Po wykonaniu tych kroków została dokonana pełna abstrakcja warstwy sprzętowej – użytkownik może dowolnie operować na nowo stworzonym systemie plików przy pomocy dowolnego eksploratora plików.

Wygląd stanowiska pracy przedstawiono na rysunku 6.5. Po prawej stronie widoczny jest mikrokontroler **CH32V305** na płytce stykowej (*breadboard*), natomiast po lewej stronie znajduje się moduł konwertera USART-USB.



Rysunek 6.5: CH32V305 z konwerterem USART-USB.

6.3. Wyzwania

Przed zakończeniem chciałbym opisać przeszkody i wymagające momenty napotkane podczas implementacji sterownika USB Mass Storage Class na mikrokontrolerze RISC-V.

Kod producenta

Kod do obsługi peryferiów i konfiguracji platformy od producenta [18] znacząco ułatwił proces rozwoju, jako że nie musiałem kontrolować elementów przez surowe dostępy do pamięci. Chociaż w praktyce ta biblioteka nie wystarczyła w całości do realizacji celu na poziomie sprzętowym, ponieważ napotkałem utrudnienia w postaci licznych rozbieżności w nazewnictwie rejestrów oraz konfiguracji pomiędzy kodem źródłowym a oficjalną dokumentacją techniczną [15]. Co więcej, przykładowy kod demonstrujący funkcjonalność kontrolera USB zawierał błędy uniemożliwiające poprawne uruchomienie.

Granice między interfejsami

Następnym wymagającym punktem było zrozumienie hierarchii interfejsów w implementacji. Specyfikacje kolejnych elementów do realizacji bardzo ogólnie opisują swoje integracje z kolejnymi krokami potoku logicznego. Nadrzędną częścią projektu było odnalezienie odpowiednich sekcji odpowiadających za interoperacyjność w specyfikacjach różnych standardów, a następnie przeniesienie tej wiedzy na kod.

Raportowanie błędów

Ostatnim godnym uwagi polem były procedury obsługi i raportowania błędów do *hosta*. Implementowanie urządzenia podrzędnego oznacza obowiązek informowania zarządzającego o potencjalnych nieprawidłowościach, łączy się to ze zrozumieniem, na której warstwie protokołu wystąpił błąd, w jaki sposób go obsłużyć wewnętrznie oraz jakie informacje należy przygotować dla *hosta* i jak je przesłać.

Rozdział 7.

Zakończenie

Realizacja tej pracy pozwoliła mi na zmierzenie się ze specyficznymi wyzwaniami, które rzadko pojawiają się w obszarze klasycznego programowania aplikacji. Praca w środowisku fizycznym, wymagająca bezpośredniego zarządzania zasobami sprzętowymi mikrokontrolera oraz synchronizacji z kontrolerami systemu operacyjnego *hosta*, stanowiła dla mnie wyjątkowe i niezwykle wartościowe doświadczenie deweloperskie. Dogłębne zrozumienie zasad działania peryferiów sprzętowych wnosi ogromną wartość do procesu projektowania systemów informatycznych, pozwalając na tworzenie znacznie bardziej efektywnego oprogramowania – również po stronie *hosta*.

Jako programista musiałem całkowicie zmienić perspektywę myślenia o kodzie – w dziedzinie systemów wbudowanych (*embedded systems*) kluczowym pojęciem determinującym sukces projektu jest **przewidywalność**. Zrozumienie, jak zapewnić płynność integracji, było jednym z najtrudniejszych zadań, z jakimi się mierzyłem.

Konfrontując wyniki z założeniami postawionymi we wstępie 1. niniejszej pracy, zrealizowałem wszystkie zaplanowane cele:

1. **Skonfigurowałem i uruchomiłem warstwę sprzętową** – dokonałem niskopoziomowej konfiguracji kontrolera USBFS, kontrolera przerwań, kontrolera pamięci FLASH oraz innych peryferiów mikrokontrolera.
2. **Zaimplementowałem stos USB oraz protokół BOT** – napisałem obsługę transferów kontrolnych i masowych, uzyskując poprawną wymianę struktur CBW oraz CSW z *hostem*.
3. **Wdrożyłem obsługę komend SCSI** – zaprogramowałem obsługę kluczowych poleceń standardu dla urządzeń blokowych umożliwiającą identyfikację i operacje na urządzeniu.
4. **Przeprowadziłem weryfikację i testy** – sprawdziłem poprawność montowania gotowego urządzenia, zapewniając przy tym pełną abstrakcję sprzętową

jego obsługi.

Wynikiem przejścia przez te kroki jest w pełni działająca emulacja zewnętrznego nośnika danych. Projekt jest również dostępny w repozytorium GitHub [28].

7.1. Możliwości rozwoju

Obecna wersja projektu ma liczne możliwości rozwoju i rozszerzenia.

Pierwszą jest użycie odpowiedniego zaplecza pamięci, użyta płytką **CH32V305** posiada interfejs komunikujący się protokołem Secure Digital Input Output (SDIO) i wspierający czytniki kart SD. To by pozwoliło na zwiększenie pojemności urządzenia i zwolniło pamięć FLASH urządzenia, ale wymagałoby implementacji nowego protokołu i obsługi zmiany nośnika pamięci (zwłaszcza przy komendach SCSI typu **PREVENT ALLOW MEDIUM REMOVAL**).

Drugą jest wsparcie dla niewspomnianego do tej pory *String Descriptors*, czyli kodowanej w **UNICODE** struktury danych podłączanej do innych deskryptorów, umożliwia ona opisywanie słowne pól takich jak nazwa produktu [26, Podsekcja 9.6.7]. Otworzy to drzwi do poprawnej rejestracji dla systemów z rodziny Windows, które definiują dodatkowe, chronione patentem *Microsoft OS Descriptors* dla rozszerzonych możliwości [13].

Następną jest obsługa szerszego zbioru komend SCSI, wspieranie bardziej zaawansowanych identyfikacji i konfiguracji urządzenia (wspieranie *vital product data*) oraz innych komend urządzenia blokowego pozwalających na lepsze zarządzanie danymi.

Ostatnią znaczącą jest wprowadzenie zapisu *write-back*, czyli wysyłanie potwierdzenia zapisu do *hosta* przed faktycznym skończeniem zapisu na medium i wewnętrzne zapewnianie spójności.

Podsumowując, zrealizowany projekt dowodzi, że implementacja niskopoziomowych protokołów interfejsu Universal Serial Bus (USB) wraz ze standardem poleceń Small Computer System Interface (SCSI) pozwala na pełną abstrakcję warstwy sprzętowej przed użytkownikiem, a także spełnia warunki poprawnego urządzenia Mass Storage Class (MSC). Dodatkowo projekt stanowi wszechstronną platformę do dalszych prac w kierunku protokołów komunikacyjnych i pamięci masowych.

Bibliografia

- [1] ANSI INCITS. *SCSI Stream Commands - 2 (SSC-2)*, 2003. 380-2003 [R2018].
- [2] Arm Limited. USB Component. <https://www.keil.com/pack/doc/mw/usb/html/index.html>, 2026. [Dostęp: 2026-04-01].
- [3] Wireshark Authors. *wireshark manual page*. The Wireshark Project, 2026.
- [4] Free Software Foundation. *screen manual page*. GNU Project, 2026. Version 5.0.0.
- [5] Future Technology Devices International Ltd. Simplified description of usb device enumeration. Technical report, FTDI, 2009. Dostęp: 2026-05-27.
- [6] Gerald Combs and contributors. *Wireshark Authors List*. The Wireshark Project, 2026.
- [7] RISC-V International. RISC-V. <https://riscv.org/>. Dostęp: 2026-05-20.
- [8] ISO/IEC. *SCSI Block Commands - 2 (SBC-2)*. 14776-322:2007.
- [9] ISO/IEC. *SCSI Primary Commands - 3 (SPC-3)*. 14776-453:2009.
- [10] Karel Zak. *dmesg manual page*. util-linux, 2026.
- [11] Karel Zak. *mount manual page*. util-linux, 2026.
- [12] Microchip Technology. *Using the USART in Asynchronous Mode*, 2001.
- [13] Microsoft. *Microsoft OS Descriptors for USB Devices*, 2024. 2024-06-27 version.
- [14] Milan Broz and Karel Zak. *lsblk manual page*. util-linux, 2026.
- [15] Nanjing Qinheng Microelectronics Co., Ltd. *CH32F/V20x_V30x_V31x Reference Manual*, 2026. Dostęp: 2026-02-02.
- [16] Nanjing Qinheng Microelectronics Co., Ltd. *QingKeV4 Microprocessor Manual*, 2026. Dostęp: 2026-02-02.
- [17] NXP Semiconductors. *I2C-bus Specification and User Manual*, 2021. Dostęp: 2026-05-25.

- [18] openwch. 32-bit Interconnected RISC-V MCU CH32V307. <https://github.com/openwch/ch32v307>, 2026.
- [19] Paul Rubin, David MacKenzie, and Stuart Kemp. *dd manual page*. GNU Coreutils, 2026.
- [20] Craig Peacock. USB in a Nutshell. <https://www.beyondlogic.org/usbnutshell/usb1.shtml>, 2026. [Dostęp: 2026-03-31].
- [21] Robert Bosch GmbH. *CAN Specification Version 2.0*, 1991. Dostęp: 2026-05-26.
- [22] Texas Instruments Inc. *KeyStone Architecture Serial Peripheral Interface (SPI)*, 2012. Dostęp: 2026-05-25.
- [23] Theodore Ts'o and Karel Zak. *fsck manual page*. e2fsprogs Project, 2026.
- [24] Thomas Sailer. *lsusb manual page*. usbutils, 2026.
- [25] USB Implementers Forum. Universal Serial Bus Mass Storage Class Bulk-Only Transport. Technical report, USB Implementers Forum, 1999.
- [26] USB Implementers Forum. Universal Serial Bus Specification Revision 2.0. Technical report, USB Implementers Forum, 2000.
- [27] Werner Almesberger, Roman Hodek and others. *mkfs.fat manual page*. dosfstools, 2026.
- [28] Kamil Zdancewicz. CH32V305-USB-MSC. <https://github.com/Kamilosok/ch32v305-usb-msc>, 2026.